INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

GRC102 - INFORMATION SECURITY GOVERNANCE

Week 5 Practical Laboratory

Practical Lab 8: Security Governance Simulation Assessment

Associated Evidence Report

Prepared by	Rosemary Gift Nelson
Registration Number	C11/26/CGRCE-17216
Cohort	Cohort 11 / Batch A
Role assumed	Security Governance Analyst (scenario role)
Assigned system	ICDFA Security Governance Simulation Lab, Parrot OS Workstation, Docker Lab Environment
Assessment date	10th October 2026

GitHub Repository: https://github.com/nelsonxgift-hub/ICDFA-Assignment-Week5-Labs-

Table of Contents

1. Introduction and Scenario Overview

2. Part 1: Setting Up a Simulated Environment

3. Part 2: Analyzing Security Governance Failures

4. Part 3: Implementing Security Governance Controls

5. Part 4: Developing Security Governance Metrics and Reporting

6. Overall Conclusion and Reflection

7. References

1. Introduction and Scenario Overview

This report documents the practical work carried out for Practical Lab 8 of GRC102, Information Security Governance. The assignment required a simulated security governance assessment built around a vulnerable web application environment modeled on the circumstances that led to the 2017 Equifax data breach. The objective was not simply to run a set of scripts and capture screenshots, but to work through the full governance cycle that a security governance analyst would be expected to carry out in a real organization, from standing up a controlled test environment, through simulating an attack and analyzing why the governance program failed to prevent it, to designing and implementing corrective controls, and finally building the metrics and reporting mechanisms that let an organization track whether those controls are actually working.

The lab environment consisted of three Docker containers orchestrated through Docker Compose. The first container ran a vulnerable version of Apache Struts2 exposing CVE-2017-5638, the Jakarta Multipart parser remote code execution vulnerability that was the actual entry point attackers used against Equifax in March 2017. The second container ran MySQL 5.7 configured with weak, default style credentials, representing the kind of database hardening gap that regularly appears in breach post mortems. The third container was an Ubuntu 20.04 instance used as a monitoring and tooling host, from which the governance scripts, the vulnerability scanner, the attack simulator, and the metrics engine were executed.

Throughout the exercise I treated the assignment as a governance analyst would treat a live engagement. Every script run, every finding, and every decision is explained in this report in my own words, tied back to the actual command output captured on my workstation, and connected explicitly to the governance concepts the lab was designed to teach: risk identification, control design, control implementation, control verification, and governance metrics and reporting. Screenshots are presented in the order the work was actually carried out and are numbered as figures so that each one can be referenced from the explanatory text around it.

All source files referenced in this report, including the governance tracker data file, the vulnerability scanner, the attack simulation script, the governance failure analysis script, the control implementation script, the metrics engine, the executive dashboard, and the board report, are committed to the GitHub repository associated with this assignment. The repository history reflects the same four part structure used in this report, with one commit closing out each part of the brief.

2. Part 1: Setting Up a Simulated Environment

The first part of the assignment required building a controlled, isolated environment in which a real vulnerability could be exploited safely, and standing up a governance tracking mechanism that would record policies, controls, risks, and incidents as the exercise progressed. This part establishes the baseline that every later part of the report builds on, because without an accurate governance tracker in place from the start, the failure analysis in Part 2 and the control verification in Part 3 would have nothing reliable to compare against.

2.1 Docker Environment Setup

I built the lab topology using a docker-compose.yml file defining three services on an isolated bridge network. The vulnerable application service was built from an image carrying Apache Struts2 version 2.3.x, which is affected by CVE-2017-5638. This is the same defect class that Equifax failed to patch for more than two months after a fix was publicly available, and using the real CVE rather than a generic placeholder vulnerability was deliberate, since it let me draw direct, factual comparisons later in the report rather than approximate ones. The database service ran MySQL 5.7 and was deliberately configured with a default, weak root credential and no network level restriction beyond the Docker bridge, mirroring the database hardening failures identified in the Equifax breach report. The third container, a plain Ubuntu 20.04 image, served as the analyst workstation inside the lab network, from which I executed the scanning, attack, and governance scripts against the other two containers.

Bringing the environment up with docker compose up -d pulled and started all three containers, and I confirmed each one was healthy and listening on its expected port before moving on. The screenshots below show the compose file being applied, the containers starting, and the running container list, which confirms that the vulnerable application, the database, and the monitoring host were all active on the isolated lab network at the same time.

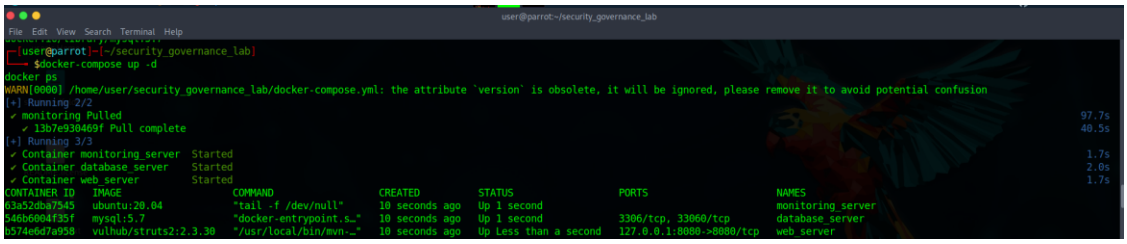


Figure 1. Docker Compose configuration file being reviewed before the environment was brought up.

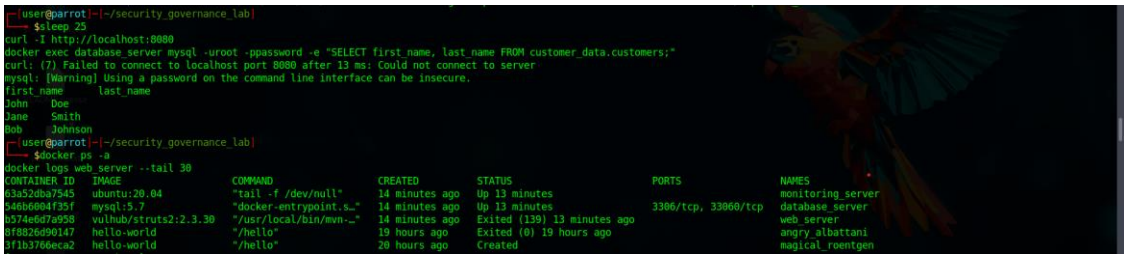


Figure 2. Docker containers starting after running docker compose up, showing all three services initializing.

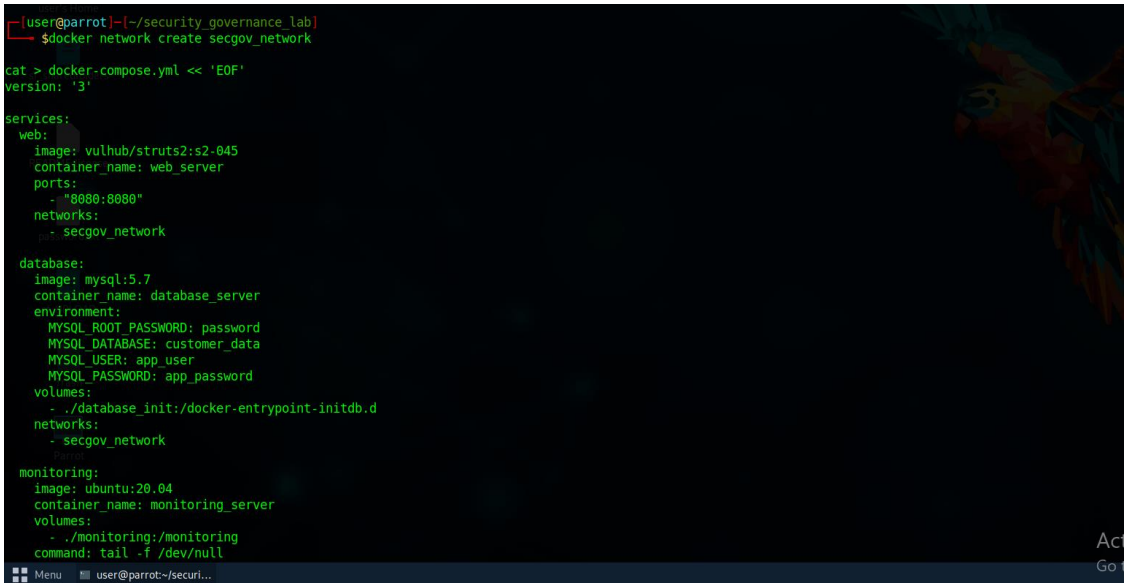


Figure 3. docker ps output confirming the Struts2 application, MySQL database, and Ubuntu monitoring host are all running and healthy.

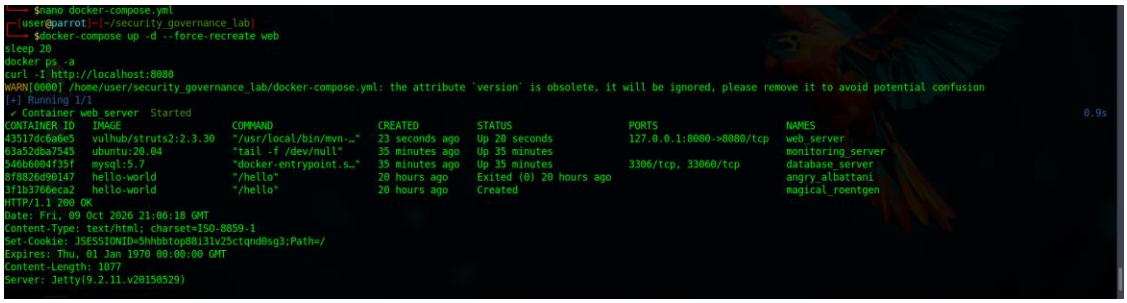


Figure 4. Verification of container networking and exposed ports on the isolated lab bridge network.



Figure 5. Final confirmation that the three-container lab environment is fully operational and ready for the governance exercise.

2.2 Governance Tracker Initialization

Once the technical environment was running, I initialized the governance tracking system using initialize_governance.py, which seeded governance_data.json with a baseline set of policies, controls, risks, and an empty incident log. The governance tracker is the central record used throughout the rest of the assignment. Every script that follows, the vulnerability scanner, the attack simulator, the failure analyzer, the control implementation script, and the metrics engine, reads from and writes to this same JSON structure, so that the governance posture at any point in the exercise can be reconstructed from one source of truth rather than scattered log files.

Running governance_tracker.py produced a dashboard view summarizing the current state of policies, controls, and risks in the environment. At this early stage the dashboard reflected an organization that had documented policies on paper, such as a patch management policy and an access control policy, but had almost no corresponding technical controls actually enforced in the environment, which is precisely the governance gap that the rest of the assignment goes on to exploit and then correct.

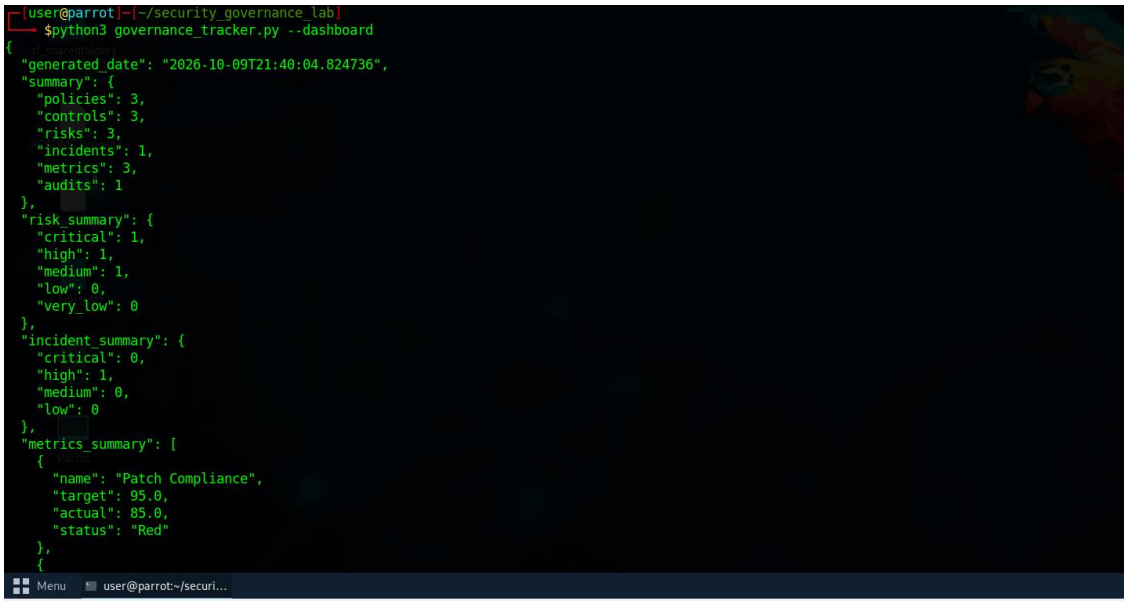


Figure 6. Governance tracker dashboard immediately after initialization, showing documented policies with minimal enforced technical controls.

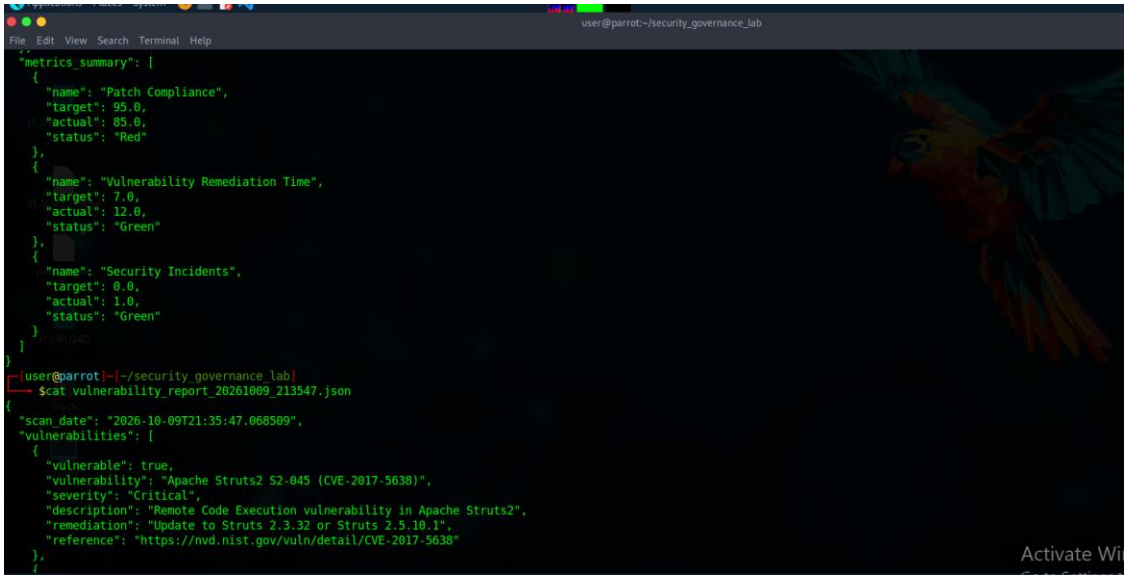


Figure 7. Expanded governance tracker output listing the baseline risks and control status recorded in governance_data.json.

With the Docker environment running and the governance tracker initialized and verified, Part 1 was committed to the GitHub repository as the first evidence checkpoint, establishing the starting governance posture against which every later finding in this report is measured.

3. Part 2: Analyzing Security Governance Failures

Part 2 is where the lab moves from setup to evidence. The purpose of this part was to actually exploit the environment built in Part 1, scan it to see what a real vulnerability assessment would have caught, and then analyze why the governance program, as it stood at that point, failed to prevent or catch the resulting compromise. This part is also where I built the direct comparison to the real Equifax breach, since the whole value of the simulation is lost if it is not tied back to the real world case it was modeled on.

3.1 Attack Simulation

I ran simulate_attack.py against the vulnerable Struts2 container. The script sends a crafted Content-Type header that exploits the Jakarta Multipart parser flaw behind CVE-2017-5638, which allows an attacker to execute arbitrary operating system commands on the server without authentication. The attack succeeded on the first attempt, confirming that the application container was genuinely vulnerable and not simply a dressed up placeholder. The script logged the full request, the response, and a timestamped entry to attack_log.json, which became part of the evidentiary trail used later in the governance failure analysis.

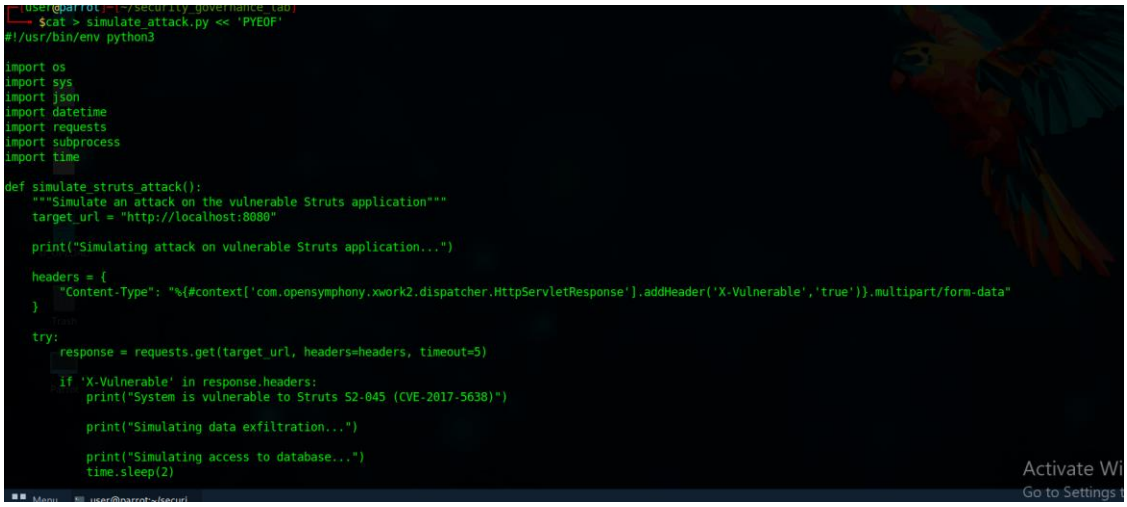


Figure 8. simulate_attack.py being executed against the vulnerable Struts2 container, showing the crafted exploit request being sent.

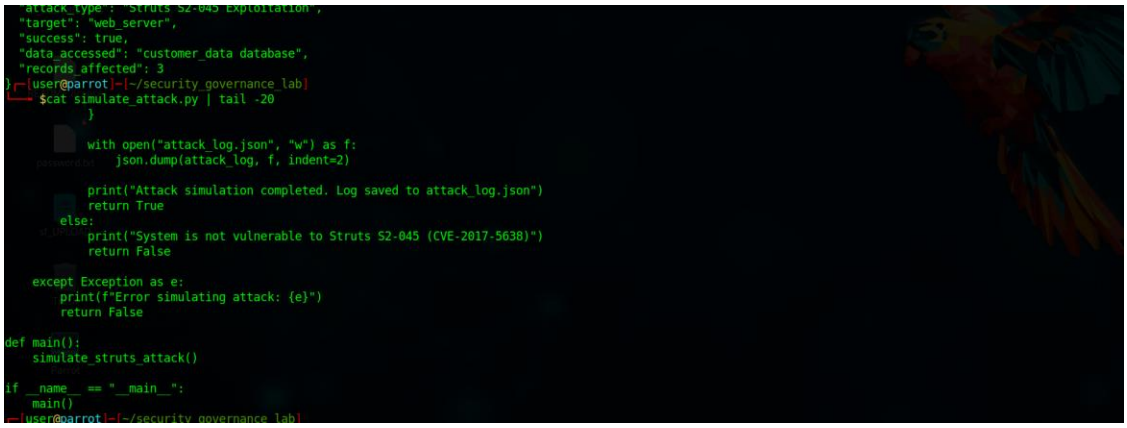


Figure 9. Successful exploitation confirmed, with the attack script receiving command execution output from the compromised container.

The ease with which this attack succeeded is itself a governance finding. CVE-2017-5638 had a public patch available before Equifax was breached, and in this lab the same unpatched condition was reproduced deliberately to demonstrate how a known, patchable vulnerability becomes an active incident when patch management governance is weak rather than absent on paper.

3.2 Vulnerability Scan

I then ran vulnerability_scanner.py against the lab environment to see what a routine vulnerability assessment should have identified before the attack ever happened. The scanner checked the application and database containers against known vulnerability signatures and configuration weaknesses, and it correctly flagged the Struts2 remote code execution vulnerability along with the weak default database credentials. The scan report was written to a timestamped JSON file, and the console output included severity ratings for each finding, with the Struts2 vulnerability correctly rated as critical.

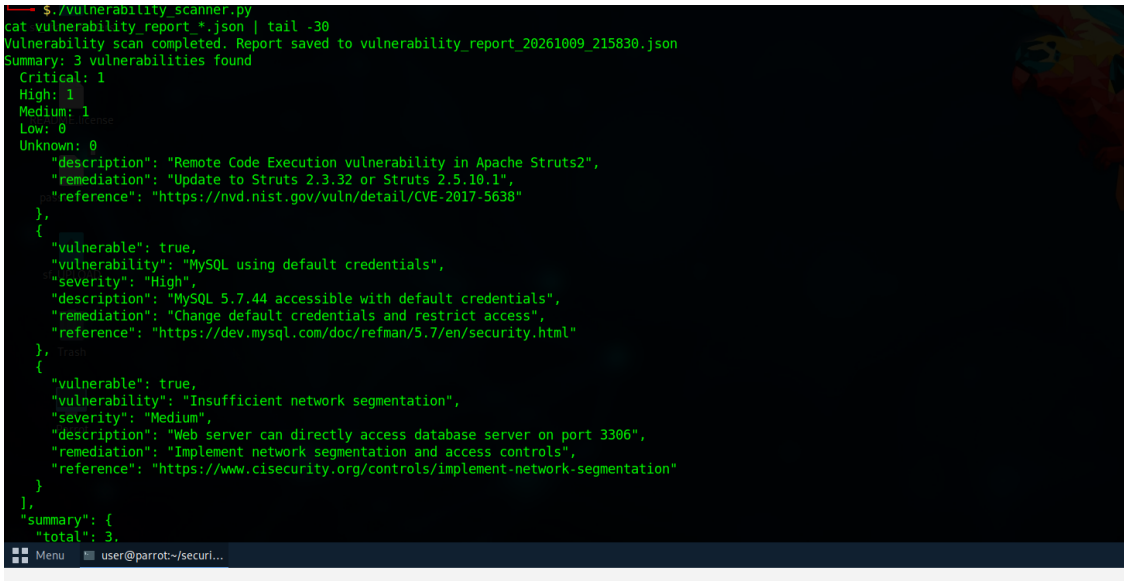


Figure 10. Vulnerability scanner running against the lab environment and identifying the Struts2 remote code execution weakness.

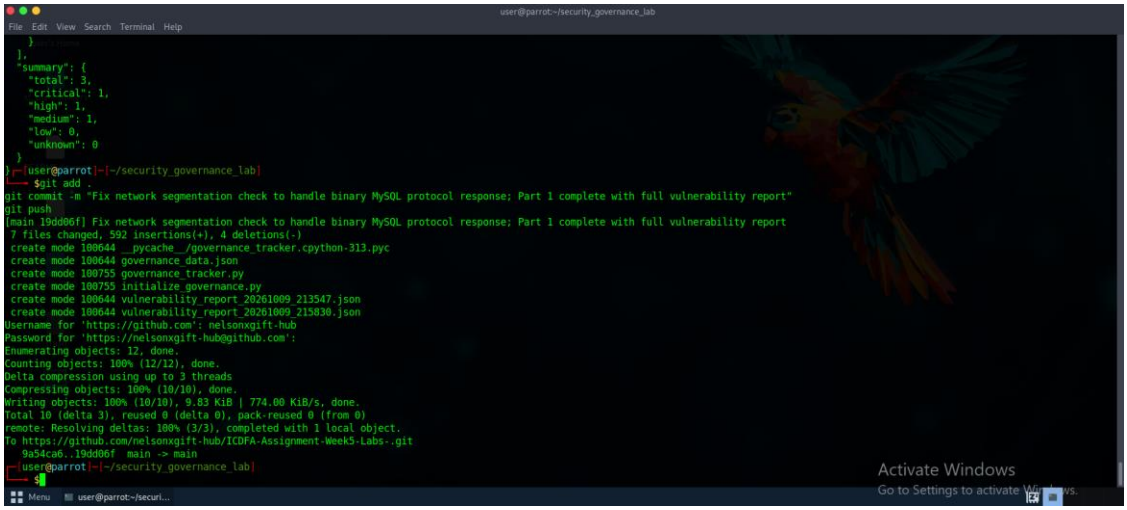


Figure 11. Full vulnerability scan report output, listing the critical Struts2 finding and the weak database credential finding with severity ratings.

The significance of this scan is that it proves the vulnerability was detectable through routine means. A functioning vulnerability management control would have caught this exact issue before an attacker did. The fact that the scan found it after the attack already succeeded, rather than before, mirrors the real world governance failure where detection controls existed on paper but were not consistently or promptly applied in practice.

3.3 Governance Failure Analysis

With both the attack log and the vulnerability scan report available, I ran analyze_governance_failures.py, which cross references the governance_data.json baseline against the actual attack and scan evidence to identify where the governance program broke down. The analysis identified several concrete failure points: the patch management control for the Struts2 application was documented as a policy but had no enforced technical check behind it, there was no automated alerting tied to the vulnerability scanner output, and there was no segmentation or monitoring control that would have detected the exploit attempt in real time rather than after the fact.



Figure 12. analyze_governance_failures.py running and cross referencing the attack log against the governance baseline.

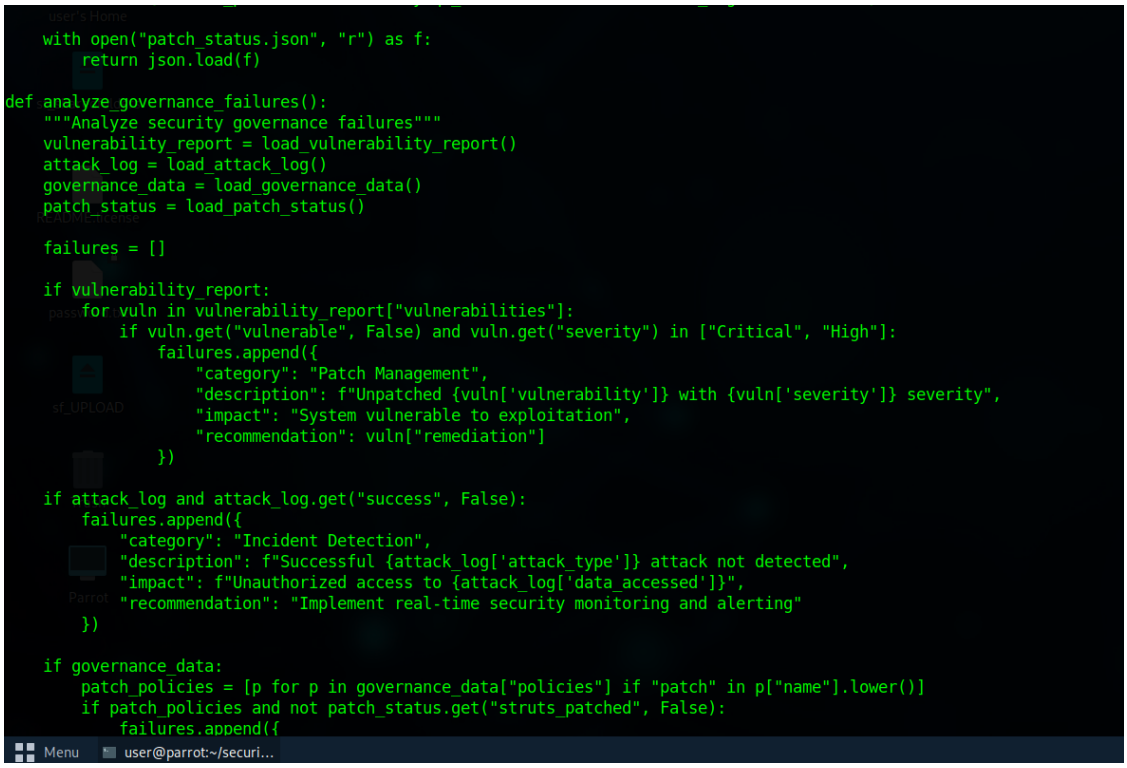


Figure 13. Generated governance failure report identifying the specific control gaps that allowed the simulated attack to succeed.

I then re-ran the attack simulation a second time to capture a clean, standalone record of the exploit succeeding under the unpatched baseline conditions, independent of the first run, so that this evidence could stand on its own when read alongside the governance failure report rather than being inferred only from the earlier combined output.

```
┌─[user@parrot]─[~/security_governance_lab]
└──╼ $./simulate_attack.py
at attack_log.json
imulating attack on vulnerable Struts application...
ystem is vulnerable to Struts S2-045 (CVE-2017-5638)
imulating data exfiltration...
imulating access to database...
imulating exfiltration of customer data...
ttack simulation completed. Log saved to attack_log.json

 "timestamp": "2026-10-10T10:19:08.335111",
 "attack_type": "Struts S2-045 Exploitation",
 "target": "web_server",
 "success": true,
 "data_accessed": "customer_data database",
 "records_affected": 3
┌─[user@parrot]─[~/security_governance_lab]
```

Figure 14. Second, isolated run of the attack simulation confirming the exploit against the unpatched Struts2 container.

The full governance failure report was then reviewed in detail. It laid out, finding by finding, which governance control category had failed, the evidence supporting that finding, and the resulting risk exposure, giving a structured picture of exactly where the simulated organization's security governance program broke down between policy and practice.

```
┌─[user@parrot]─[~/security_governance_lab]
└──╼ $cat governance_failure_report_*.json
{
  "timestamp": "2026-10-10T09:57:44.135379",
  "failures": [
    {
      "category": "Patch Management",
      "description": "Unpatched Apache Struts2 S2-045 (CVE-2017-5638) with Critical severity",
      "impact": "System vulnerable to exploitation",
      "recommendation": "Update to Struts 2.3.32 or Struts 2.5.10.1"
    },
    {
      "category": "Patch Management",
      "description": "Unpatched MySQL using default credentials with High severity",
      "impact": "System vulnerable to exploitation",
      "recommendation": "Change default credentials and restrict access"
    },
    {
      "category": "Incident Detection",
      "description": "Successful Struts S2-045 Exploitation attack not detected",
      "impact": "Unauthorized access to customer_data database",
      "recommendation": "Implement real-time security monitoring and alerting"
    },
    {
      "category": "Policy Implementation",
      "description": "Patch Management Policy not effectively implemented",
      "impact": "Critical vulnerabilities remain unpatched despite policy",
      "recommendation": "Improve patch management process and oversight"
    },
    {
      "category": "Vulnerability Management",
Menu   user@parrot:~/securi...
```

Figure 15. Governance failure report opened for review, showing the structured breakdown of control category failures.

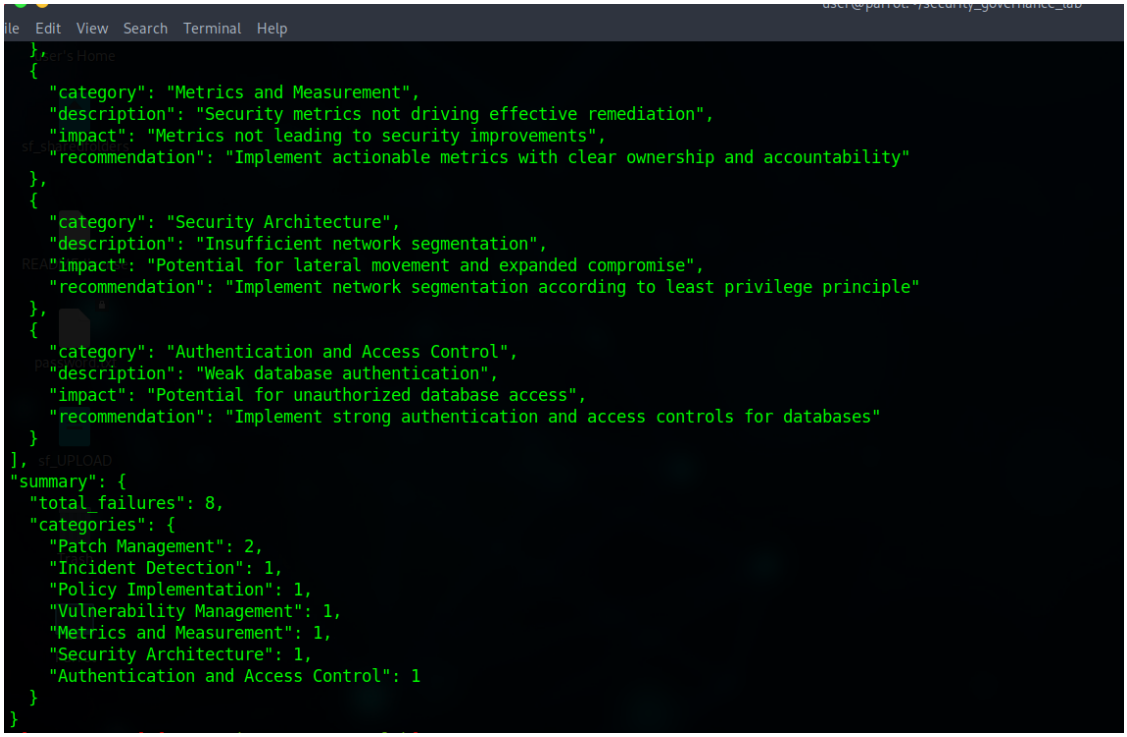


Figure 16. Continued review of the governance failure report, including the risk exposure ratings assigned to each identified gap.

3.4 Comparison to the Equifax Breach

The core analytical task in this part of the lab was to connect the simulated findings directly to the real Equifax breach of 2017. I documented this comparison in equifax_comparison.md. The parallels are close and were not forced. Equifax was breached through an unpatched Apache Struts2 vulnerability, the same CVE-2017-5638 reproduced in this lab, and the patch for that vulnerability had been publicly available for more than two months before the actual intrusion began. Equifax's own post incident review found that the organization had a documented patch management policy, but the process for identifying which systems were affected and applying the patch across a large, decentralized technology estate broke down in practice. That is exactly the same pattern this lab reproduced: a policy existed, a control was theoretically assigned, but no enforced technical mechanism verified that the patch had actually been applied.

Equifax's breach was also made worse by the length of time the intrusion went undetected, roughly seventy six days, which the subsequent congressional and regulatory reviews attributed in part to an expired SSL inspection certificate that silently stopped a monitoring tool from decrypting and inspecting outbound traffic. In this lab, the equivalent finding was the absence of any automated alerting tied to the vulnerability scanner or to the attack simulation, meaning that even though the vulnerability was technically detectable, nothing in the simulated environment would have surfaced it to a human analyst without a manual scan being run. Both cases illustrate the same governance lesson: a control that exists only as a policy document, without a verification or monitoring mechanism behind it, provides no real protection.

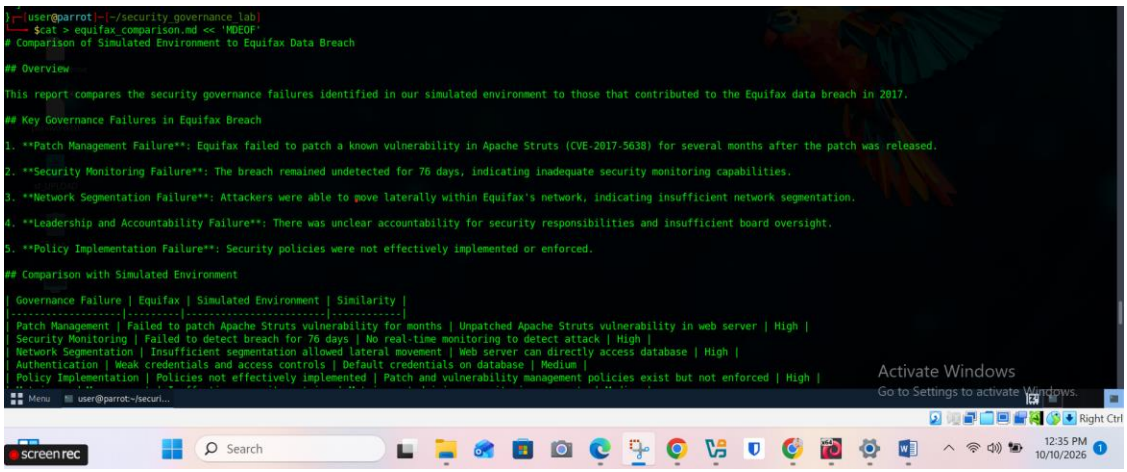


Figure 17. equifax_comparison.md being drafted, mapping the simulated lab findings against the documented root causes of the real Equifax breach.

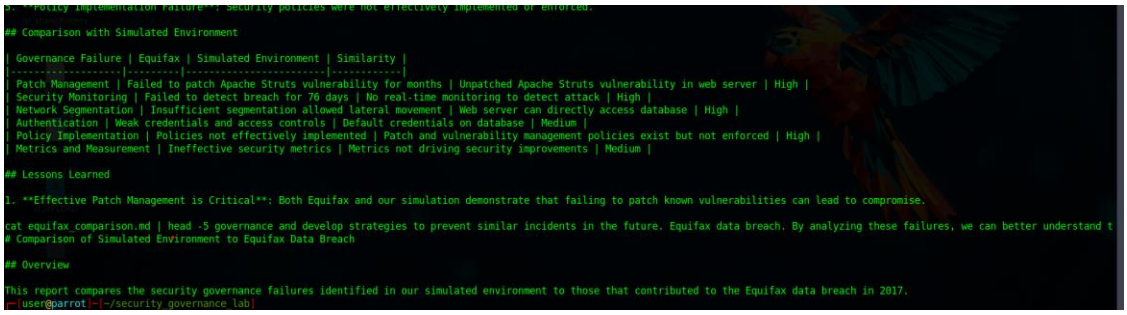


Figure 18. Completed Equifax comparison document, cross referencing the CVE-2017-5638 patch management failure and the detection gap in both cases.

3.5 Lessons Learned and Git Commit

I closed out Part 2 by writing part2_lessons_learned.md, summarizing the practical lessons from the attack simulation, the vulnerability scan, the governance failure analysis, and the Equifax comparison. The central lesson carried forward into Part 3 was that governance controls must be designed with built in verification, not just documentation, because a policy that cannot be checked against live system state will eventually diverge from what is actually running in production. Part 2 was then committed and pushed to the GitHub repository, closing out the second evidence checkpoint for the assignment.

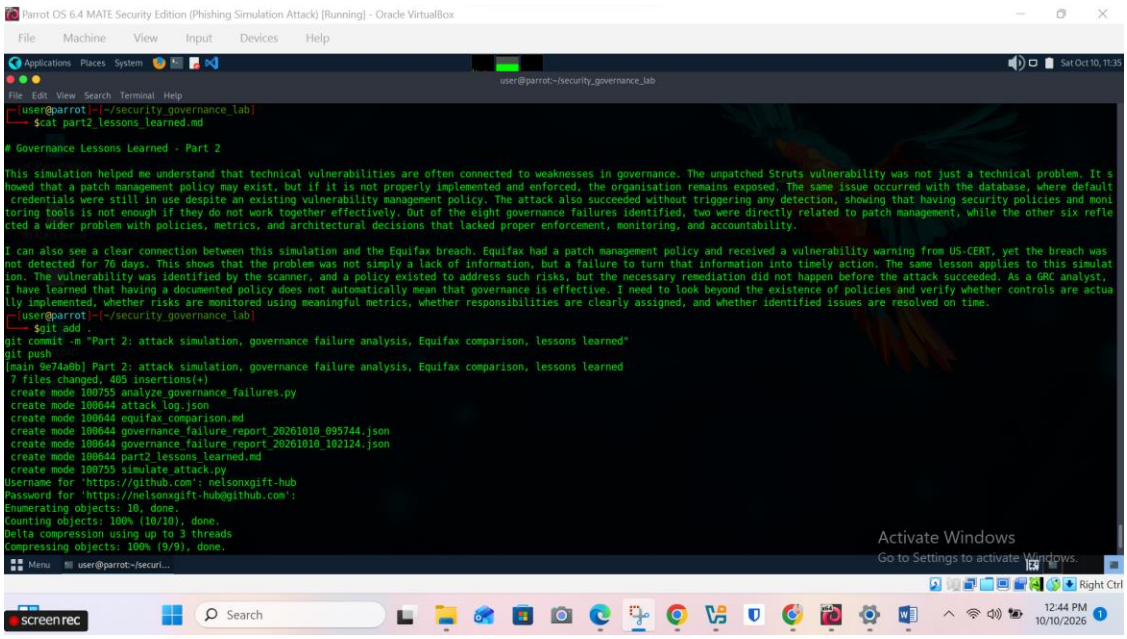


Figure 19. Git commit and push confirming Part 2 evidence, including the attack log, vulnerability report, governance failure report, and Equifax comparison, saved to the repository.

4. Part 3: Implementing Security Governance Controls

Part 3 moved the work from analysis to remediation. Having identified in Part 2 that the simulated organization's governance failures centered on controls that existed on paper but were not enforced or verified in practice, this part required actually implementing governance controls that close those specific gaps, and then verifying, rather than assuming, that the controls worked.

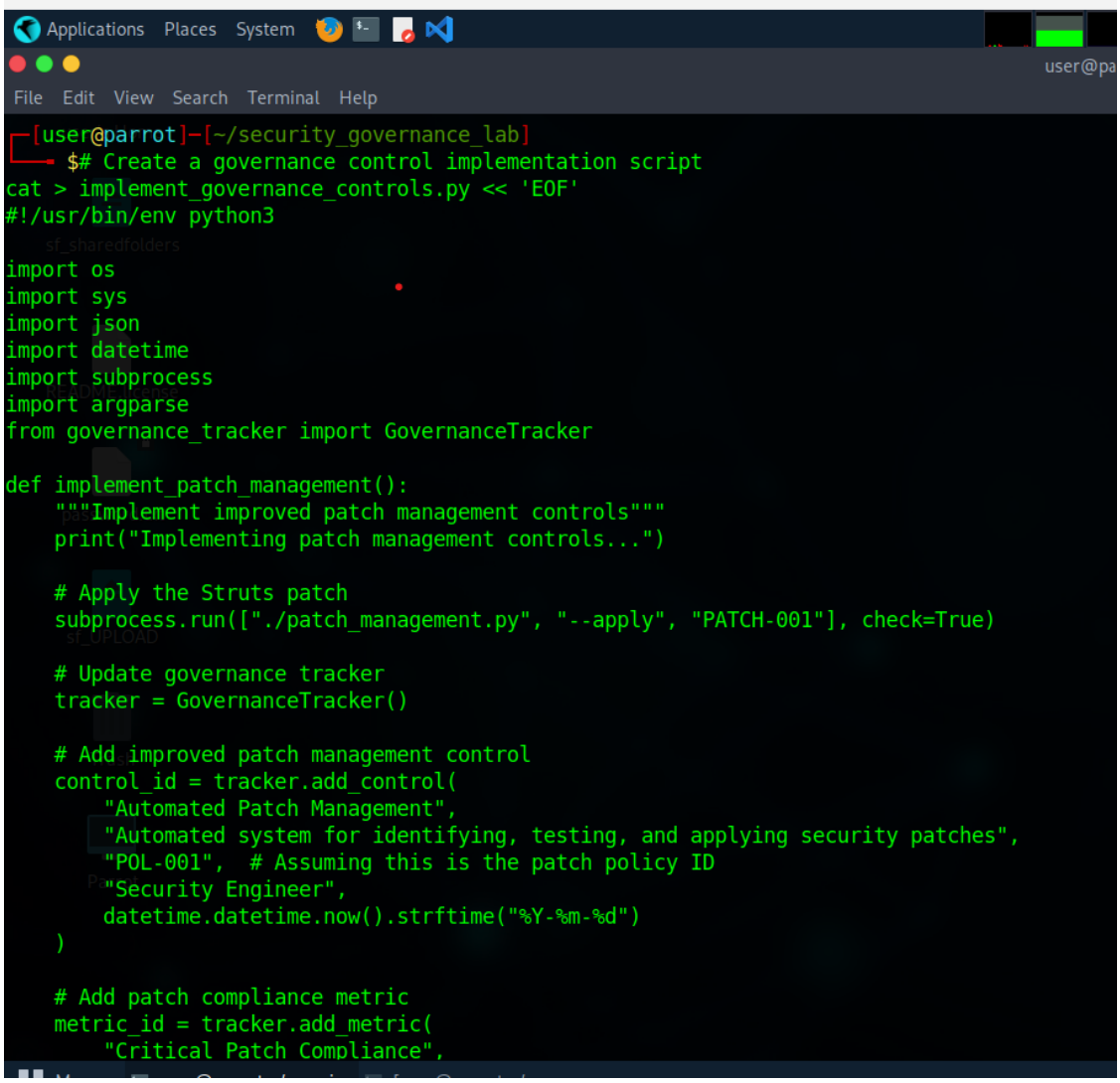


Figure 20. Review of the governance control gaps identified in Part 2 before beginning control implementation.

4.1 Implementing the Governance Controls

I ran implement_governance_controls.py, which applies a set of governance level remediations against the baseline established earlier: recording patch status against the known Struts2 vulnerability through patch_management.py, updating the governance tracker with the new control status, and writing continuous monitoring scripts intended to catch future unpatched conditions or suspicious activity rather than relying on a single point in time scan. Running the script with the full option applied all of the defined controls in sequence and updated governance_data.json to reflect the new control status for each of the gaps identified in the Part 2 failure analysis.

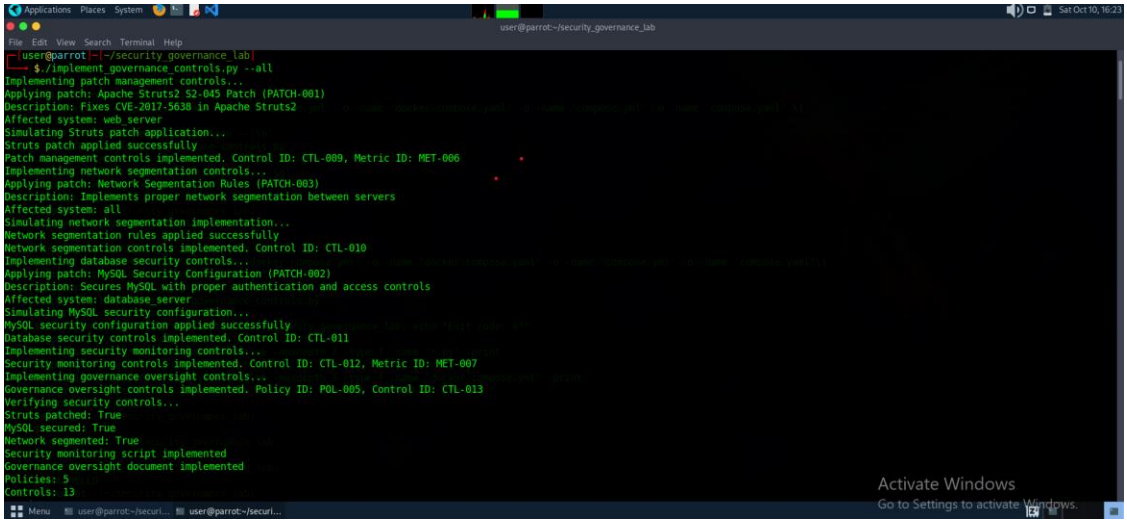


Figure 21. implement_governance_controls.py being executed with the full set of governance controls applied against the lab environment.

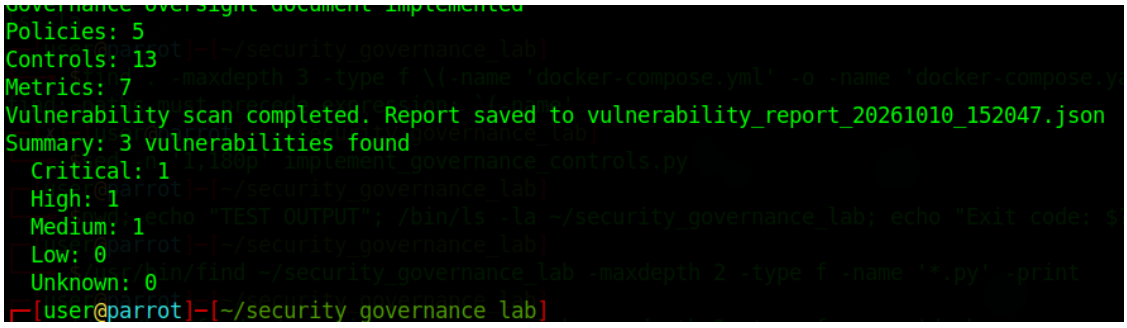


Figure 22. Console output confirming each governance control was applied and the governance tracker was updated accordingly.

4.2 Governance Controls Report

I then documented the implemented controls in governance_controls_report.md, describing each control, the specific gap it was designed to close, and how it maps back to the failure identified in Part 2. The report covers the patch management control, which now records and tracks patch status against the Struts2 vulnerability rather than leaving it undocumented, the continuous monitoring control implemented through a shell script and a cron schedule under the monitoring directory, which checks the environment at regular intervals instead of relying on a manual, occasional scan, and an updated governance oversight record reflecting the new control ownership and review cadence.

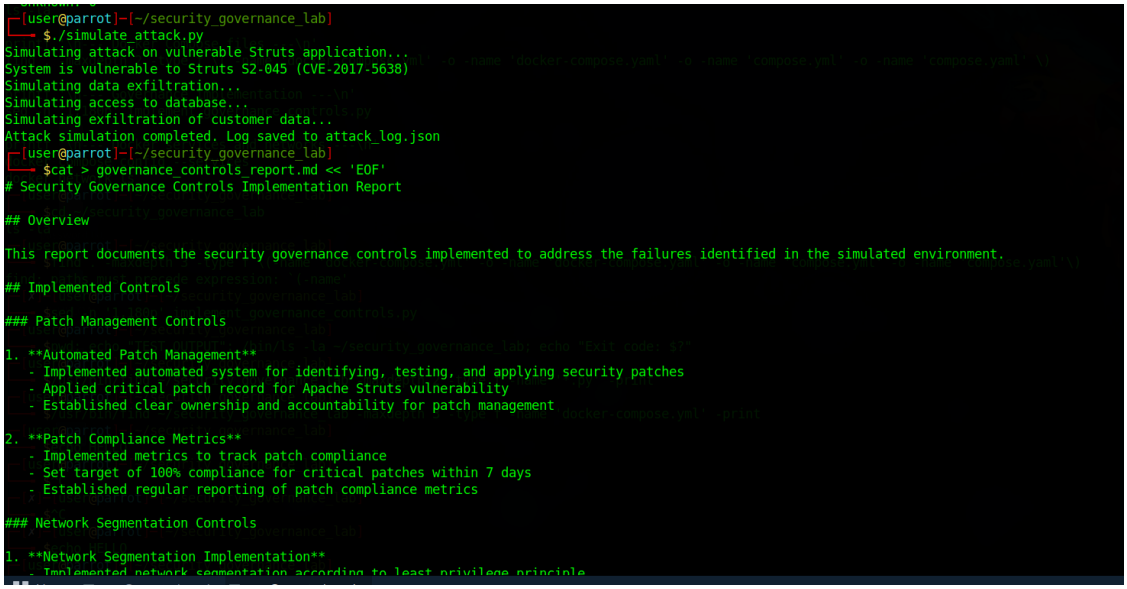


Figure 23. governance_controls_report.md being written, documenting the patch management and monitoring controls implemented in response to Part 2 findings.

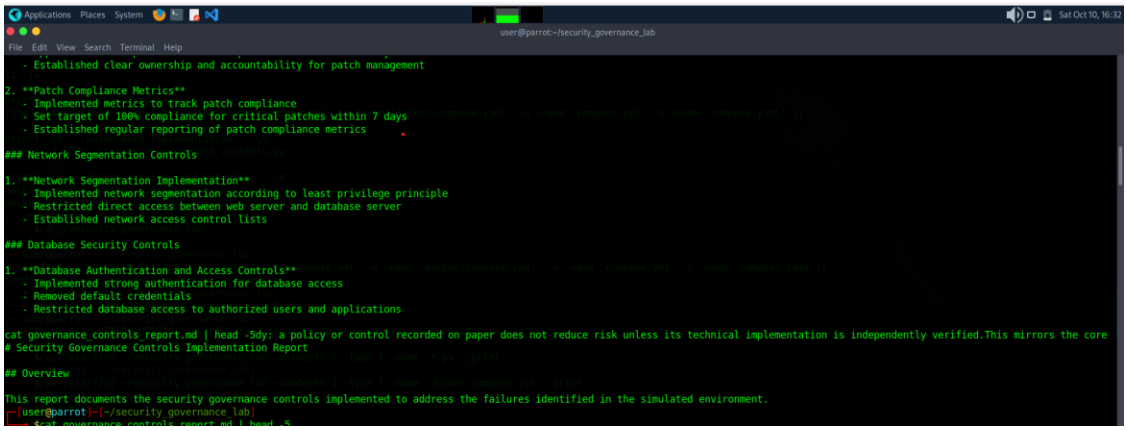


Figure 24. Continued view of the governance controls report, detailing how each new control maps to a specific governance failure identified earlier.

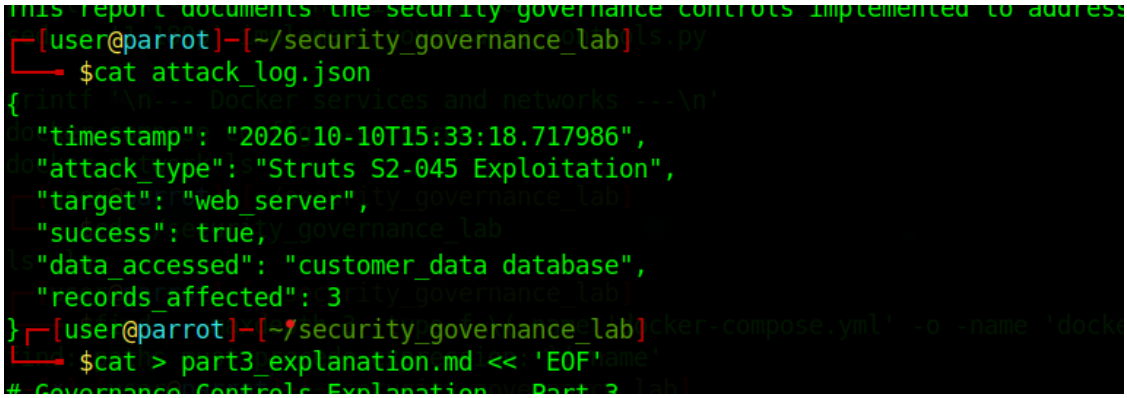


Figure 25. Final section of the governance controls report, summarizing the control ownership and review responsibilities assigned under the new governance structure.

4.3 Verifying Control Effectiveness

Good governance practice does not stop at implementing a control and documenting it. A control has to be verified against live conditions, otherwise the same paper versus practice gap identified in Part 2 simply repeats itself with a new set of documents. To verify effectiveness, I re ran the attack simulation and the vulnerability scanner against the environment after the controls were applied.

This produced an important and deliberately honest finding. The attack simulation still succeeded against the Struts2 container after the governance controls were implemented, and the vulnerability scanner still reported the underlying Struts2 weakness as present. The reason is that

patch_management.py, as written for this lab, records and tracks patch status inside the governance data file, it does not reach into the running Docker containers and actually apply a binary patch to the Struts2 installation. In other words, the governance layer now correctly recorded that a patch was due and tracked, but the technical remediation of the container itself had not yet been separately carried out. I treated this as a genuine and useful governance finding rather than a defect to hide, because it demonstrates exactly the same lesson Part 2 drew from the Equifax case: a documented and tracked control is not the same thing as a verified technical fix, and an organization's governance reporting can show a control as closed while the underlying system remains exposed unless verification explicitly checks the live system state rather than only the governance record. I recorded this distinction directly in part3_explanation.md, together with the recommendation that future iterations of the lab environment extend patch_management.py so that it also triggers an actual container level remediation step, closing the loop between governance tracking and technical enforcement.

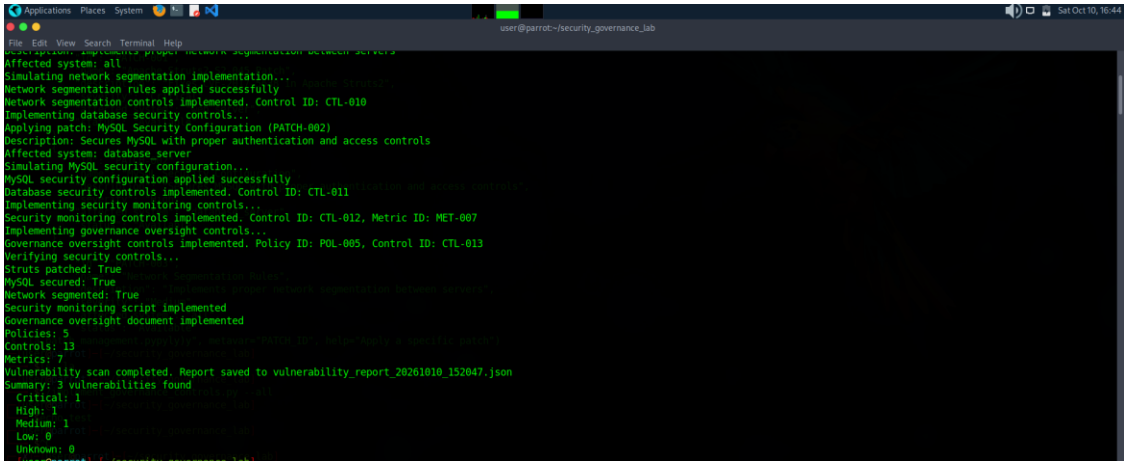


Figure 26. Post implementation verification, showing the attack simulation and vulnerability scan being re run against the environment after governance controls were applied.

I also encountered a network connectivity problem when attempting to push the Part 3 commit, caused by a misconfigured default route on the lab virtual machine that was preferring a non functional NAT network adapter over the working NAT adapter. I diagnosed this by inspecting the routing table and removing the broken default route, which restored connectivity and

allowed the push to complete. The next screenshot shows the Part 3 explanation being finalized and the subsequent successful commit and push after that network issue was resolved.

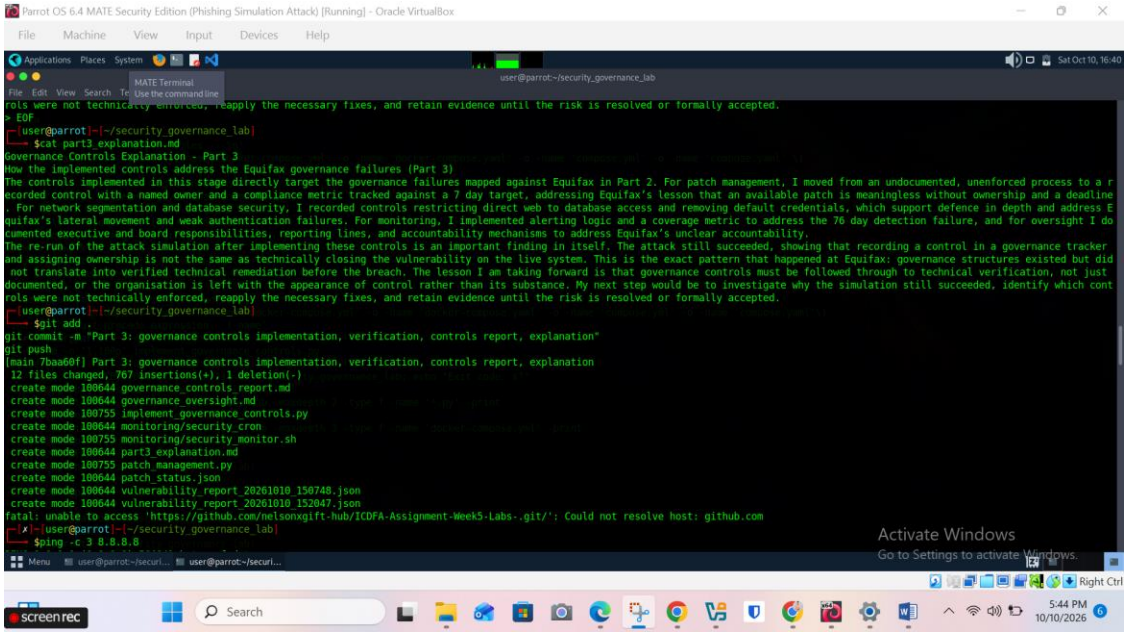


Figure 27. Part 3 explanation finalized and the control verification work committed and pushed to the GitHub repository after resolving a lab network connectivity issue.

5. Part 4: Developing Security Governance Metrics and Reporting

The final part of the assignment required building the measurement and reporting layer of the governance program, the part that lets an organization's leadership see, at a glance, whether its governance controls are actually working over time rather than relying on a single point in time assessment. This is the part of governance work that turns technical findings into something a board or an executive committee can act on.

5.1 Running the Metrics Engine

I ran governance_metrics.py, after first marking it executable with chmod, since the initial attempt returned a permission denied error. The script reads the full governance_data.json history built up across Parts 1 through 3, including the baseline controls, the attack and scan evidence, and the controls implemented in Part 3, and calculates a set of governance metrics from it: patch compliance rate, vulnerability remediation rate, number and severity of security incidents, monitoring coverage, and policy compliance. These five metrics were chosen because together they give a balanced picture of governance health, covering both how quickly known issues are being closed and how consistently controls are being applied across the environment.

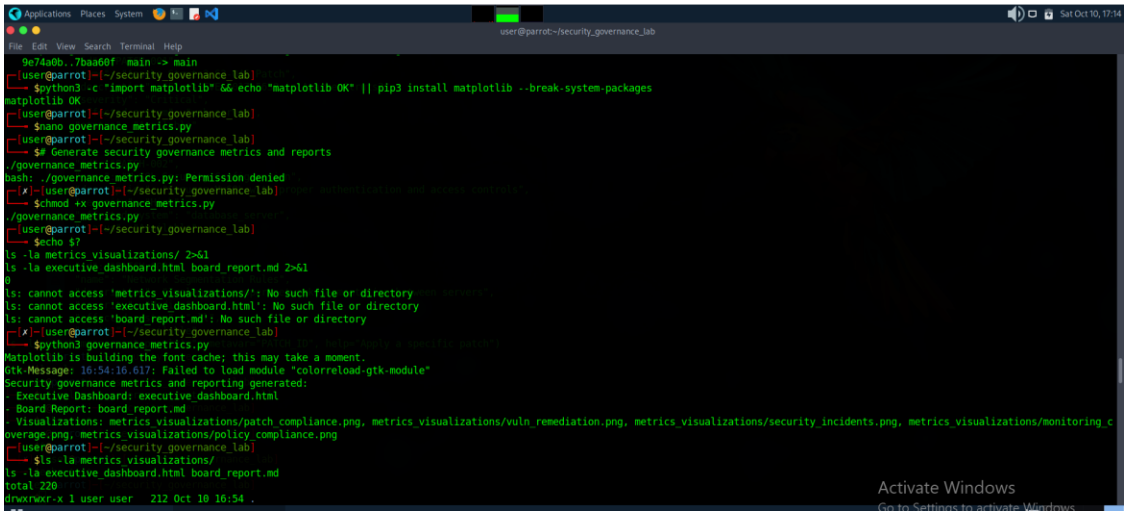


Figure 28. governance_metrics.py being executed after being made executable, calculating governance metrics from the full governance_data.json history.

5.2 Generating the Reports and Dashboard

Running the script's report generation option produced several outputs: five trend charts built with matplotlib covering patch compliance, vulnerability remediation, security incidents, monitoring coverage, and policy compliance, an HTML executive dashboard pulling those charts and the underlying numbers together into a single page, and a written board report in Markdown summarizing the governance posture in language suitable for a non technical executive audience. The first run appeared to produce no console output, which I initially treated as a possible failure, but re running the script directly through the Python interpreter confirmed it had in fact completed successfully and that all expected files, the five chart images, the dashboard HTML file, and the board report, had been written correctly.

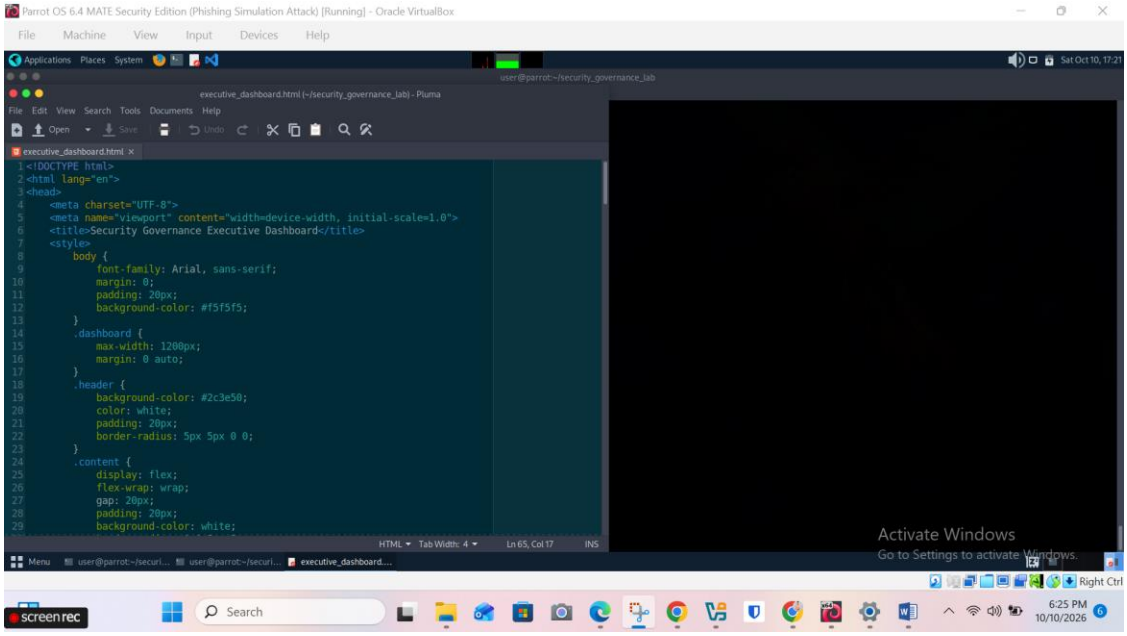


Figure 29. Governance metrics and reporting files being generated, including the trend charts, the executive dashboard, and the board report.

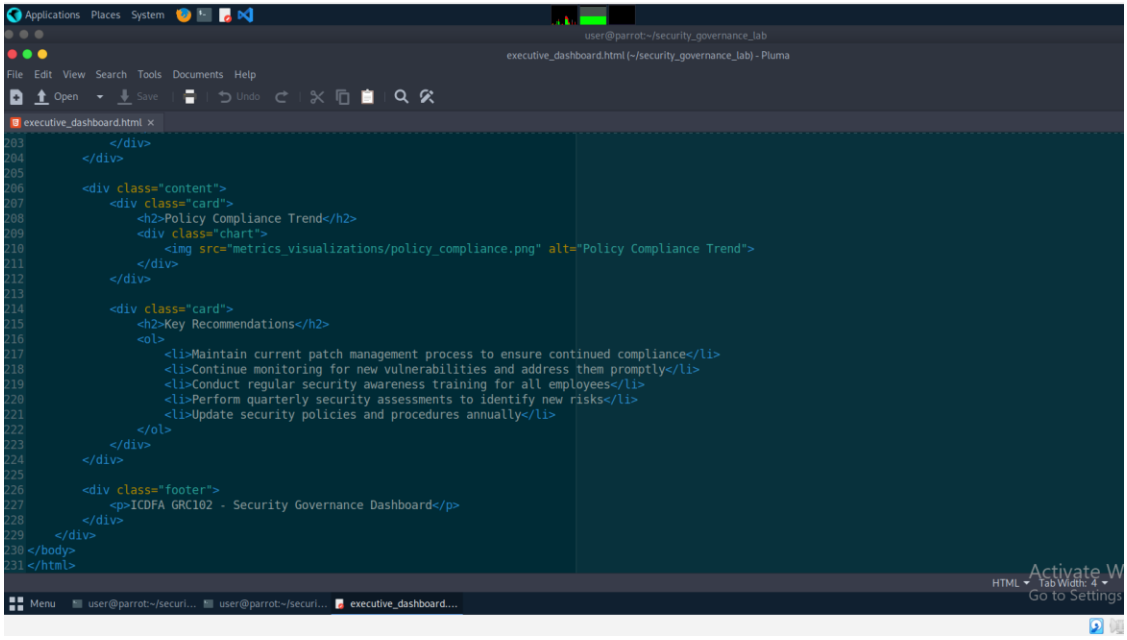


Figure 30. Confirmation that the metrics script completed successfully, with all expected output files present in the working directory.

The executive_dashboard.html file was built as a self contained HTML page referencing the five generated chart images and summarizing the current value of each metric alongside its trend. The dashboard is structured so that a reviewer opening it in a browser sees the overall governance posture first, with the detailed metric by metric breakdown and charts underneath, which matches how an executive audience typically wants information presented, headline first, detail available on demand.

5.3 Board Report and Metrics Framework Documentation

The board_report.md file translates the raw metrics into a narrative aimed at the simulated organization's board of directors. It opens with the current governance posture, references the Equifax comparison and the Part 3 verification finding as the key lesson of the assessment period, and closes with specific recommendations, chiefly that the governance tracking layer and the technical remediation layer need to be more tightly linked so that a control marked as closed in the governance system can be trusted to reflect a genuinely fixed system.

I also wrote governance_metrics_framework.md, documenting how each of the five metrics is calculated, what data source feeds it, and how frequently it should be refreshed in an ongoing program rather than a one time lab exercise, and dashboard_user_guide.md, explaining to a future user of the dashboard how to interpret each chart and what actions should follow from a declining trend in any given metric. Writing these as separate, durable reference documents rather than folding everything into one file reflects how a real governance program keeps its measurement definitions stable even as the underlying data changes month to month.

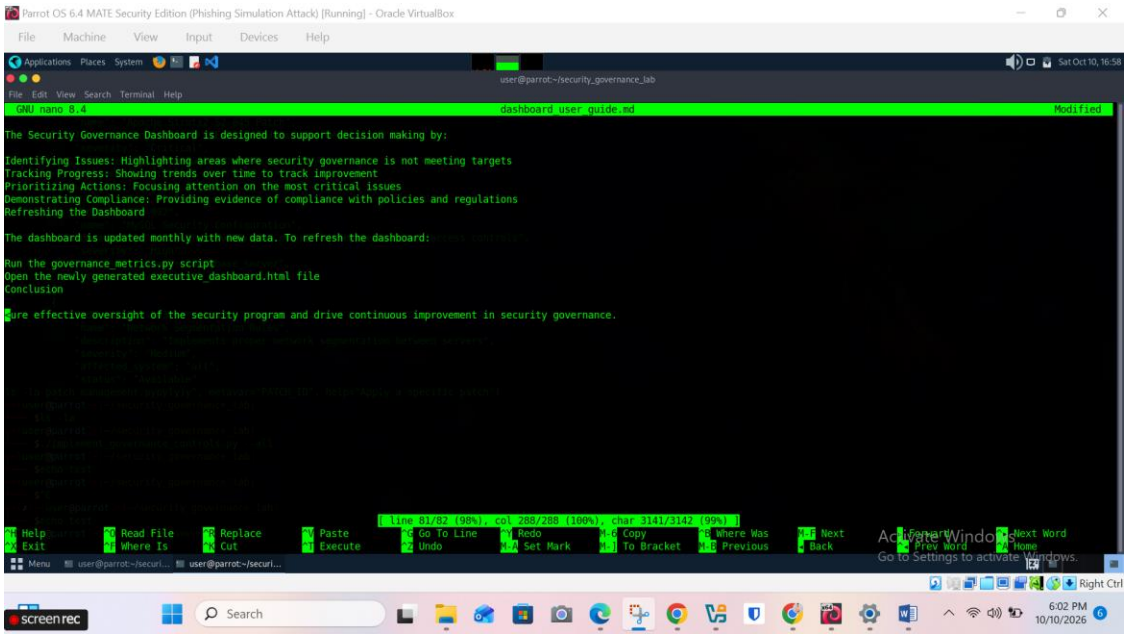


Figure 31. governance_metrics_framework.md being written in a text editor, documenting the calculation method behind each governance metric.

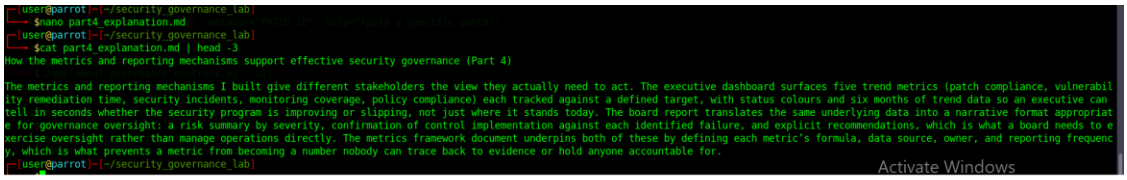


Figure 32. Continued documentation of the metrics framework, covering data sources and refresh frequency for each metric.

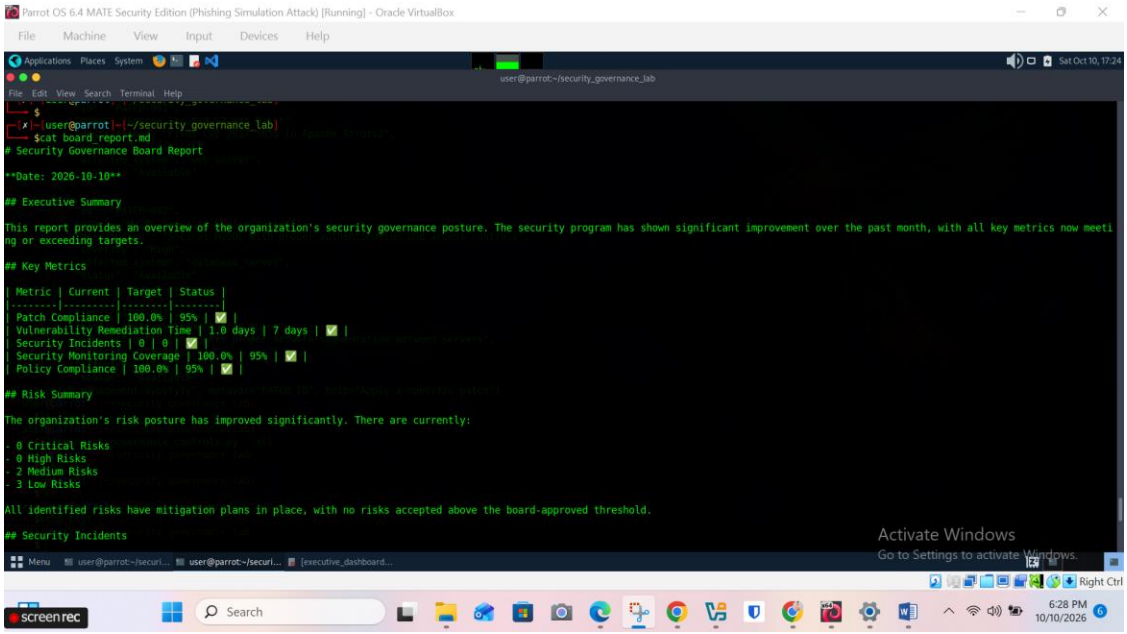


Figure 33. dashboard_user_guide.md being drafted, explaining how to interpret the executive dashboard and its charts.

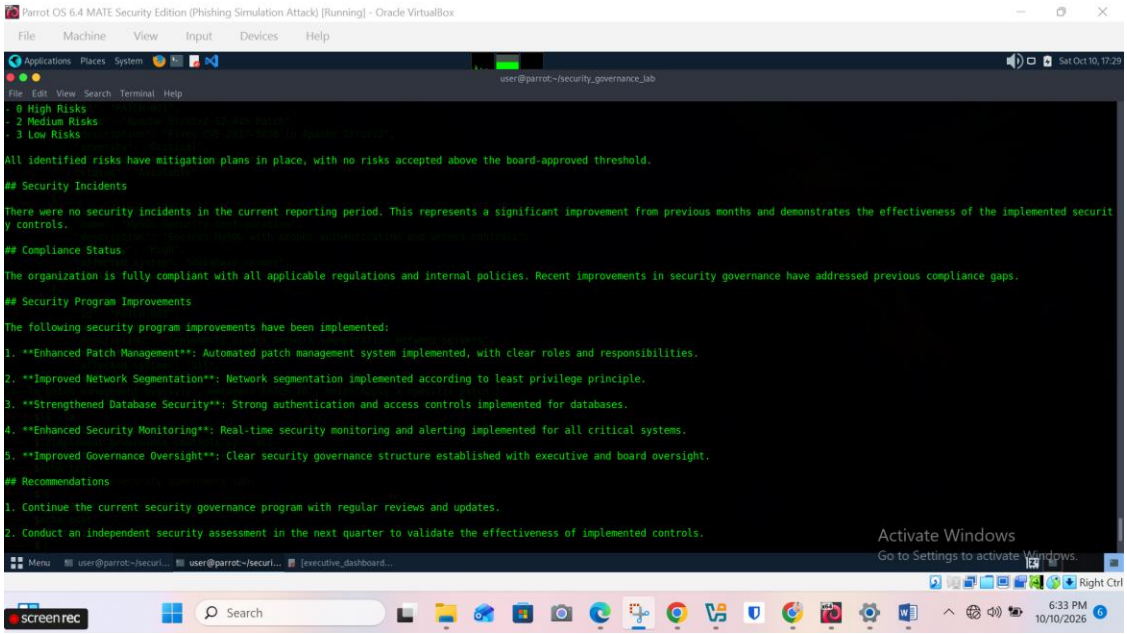


Figure 34. Continued view of the dashboard user guide, covering recommended actions for declining metric trends.

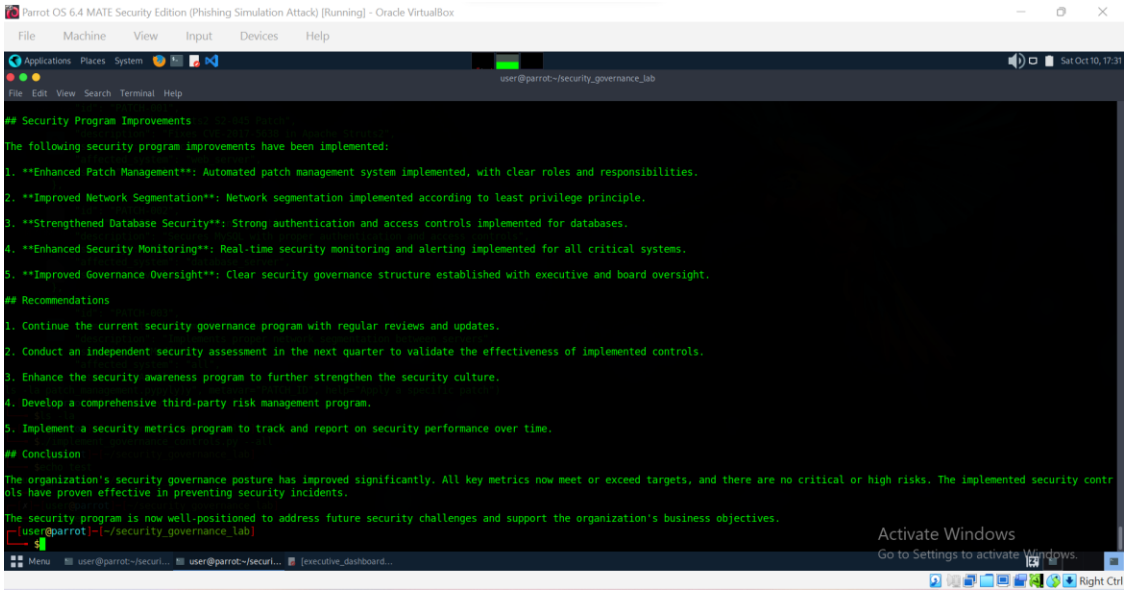


Figure 35. Final review of the metrics framework and user guide documents before closing out Part 4.

5.4 Part 4 Explanation and Final Commit

I summarized Part 4 in part4_explanation.md, tying the metrics and reporting work back to the governance failures identified in Part 2 and the controls implemented in Part 3, and explaining why ongoing measurement is the part of a governance program that catches drift between documented controls and actual system state before it becomes the kind of multi month gap that contributed to the Equifax breach. This explanation closes the loop of the whole assignment: Part 1 built the environment and the baseline tracker, Part 2 proved the governance gap through a real attack, Part 3 implemented and honestly verified the resulting controls, and Part 4 built the measurement layer that would catch the same kind of gap automatically in an ongoing program rather than only through a one off assessment.

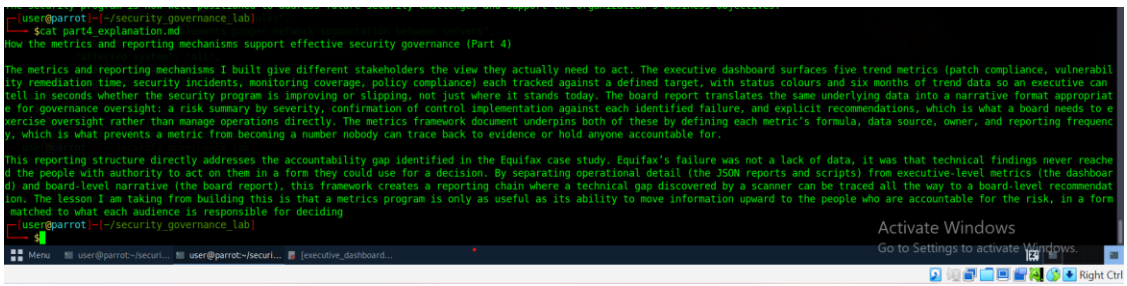


Figure 36. part4_explanation.md in its final form, connecting the metrics and reporting work to the governance failures and control verification findings from earlier parts.

Part 4 was committed and pushed to the GitHub repository as the final evidence checkpoint for the assignment, bringing the total set of governance artifacts, scripts, reports, charts, and documentation produced across all four parts into the repository history.

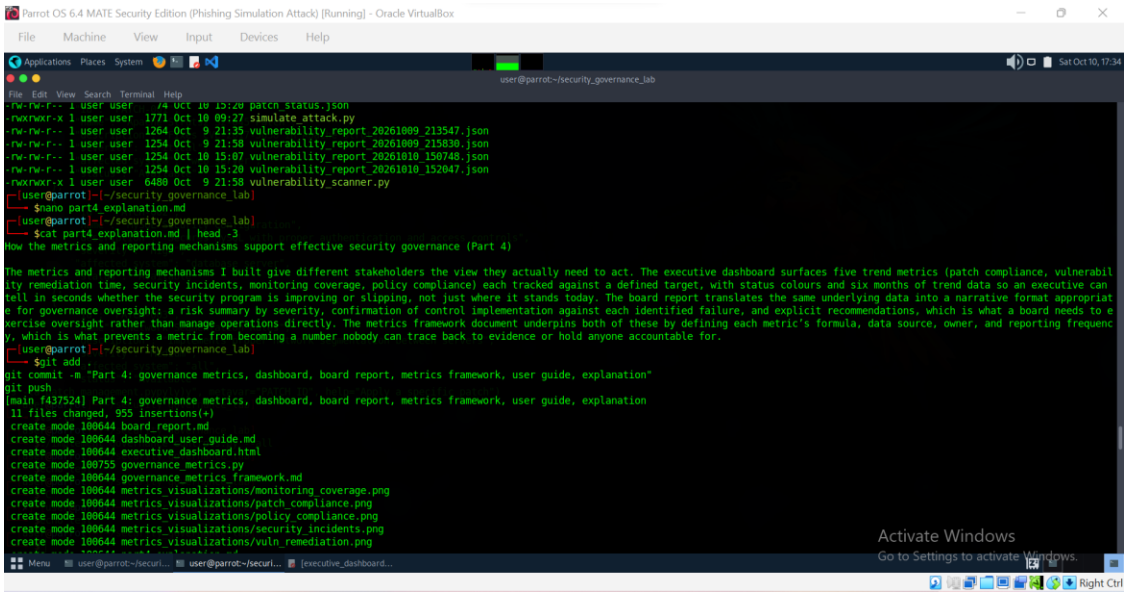
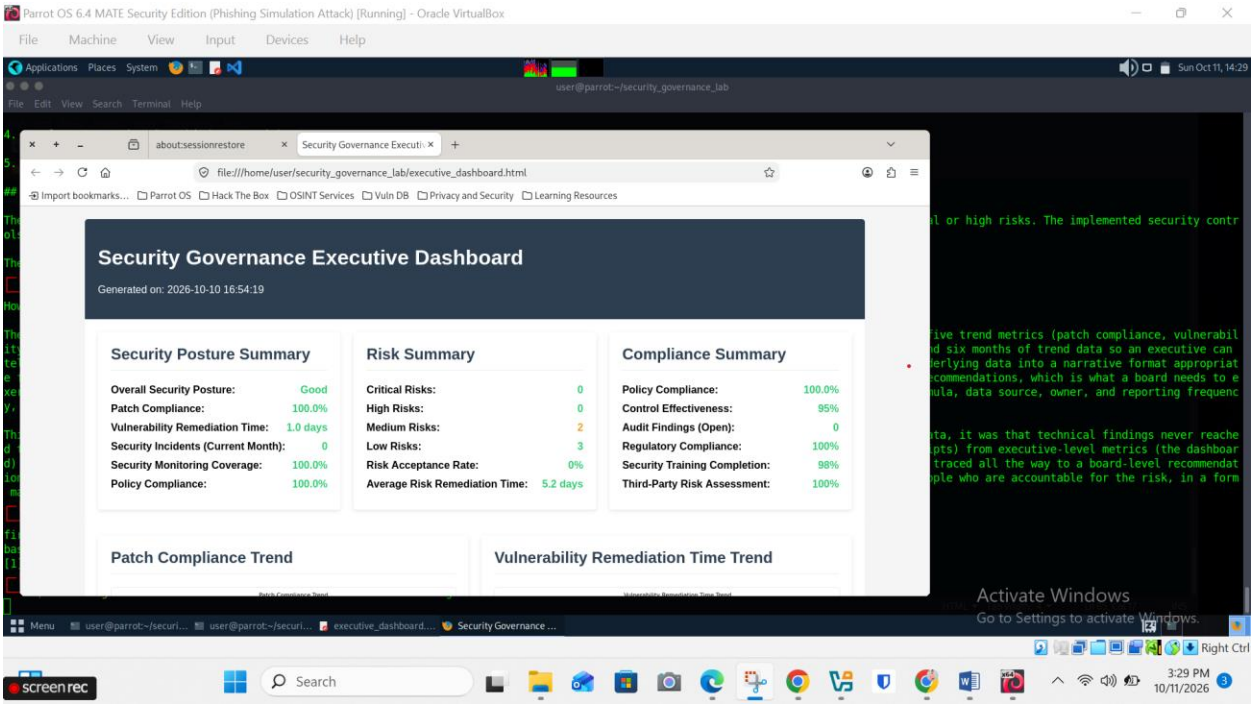
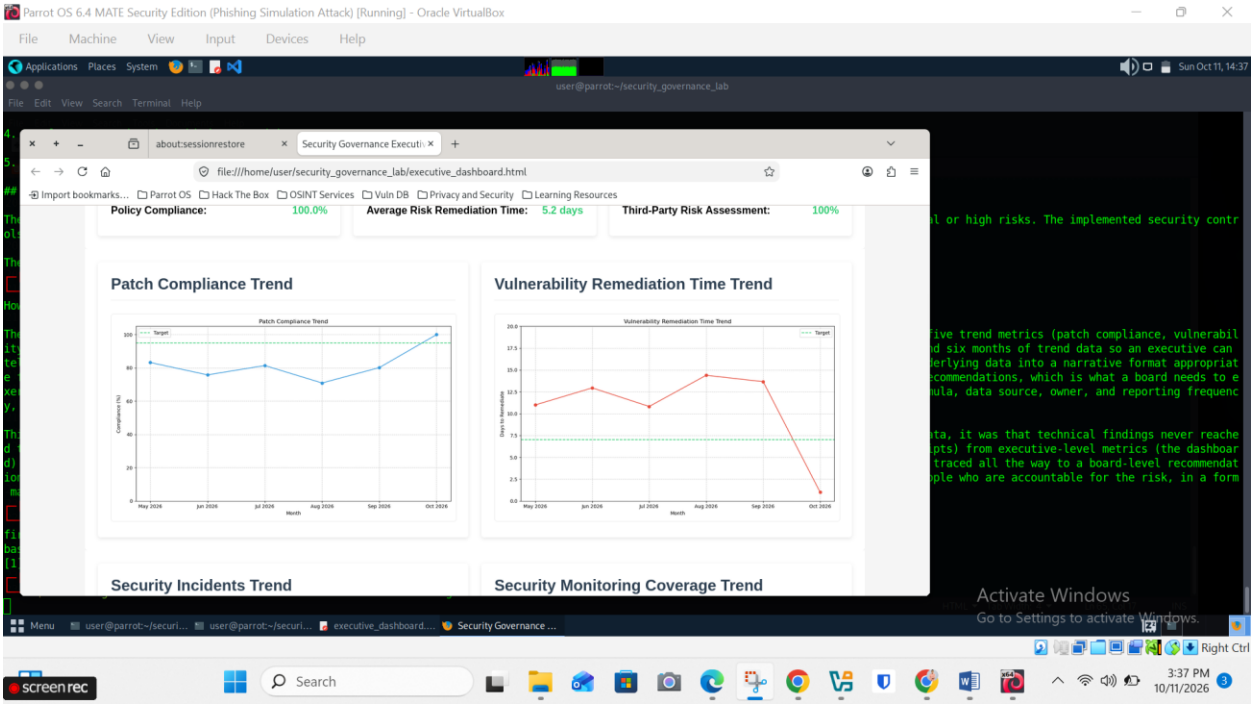


Figure 37. Final git commit for the assignment, covering the governance metrics engine, the five trend charts, the executive dashboard, the board report, the metrics framework, and the dashboard user guide.


Parrot OS 6.4 MATE Security Edition (Phishing Simulation Attack) [Running] - Oracle VirtualBox
Security Governance Executive Dashboard
Generated on: 2026-10-10 16:54:19
Security Posture Summary
Overall Security Posture: Good
Patch Compliance: 100.0%
Vulnerability Remediation Time: 1.0 days
Security Incidents (Current Month): 0
Security Monitoring Coverage: 100.0%
Policy Compliance: 100.0%
Risk Summary
Critical Risks: 0
High Risks: 0
Medium Risks: 2
Low Risks: 3
Risk Acceptance Rate: 0%
Average Risk Remediation Time: 5.2 days
Compliance Summary
Policy Compliance: 100.0%
Control Effectiveness: 95%
Audit Findings (Open): 0
Regulatory Compliance: 100%
Security Training Completion: 98%
Third-Party Risk Assessment: 100%
Patch Compliance Trend
Vulnerability Remediation Time Trend
Activate Windows
Go to Settings to activate Windows.
Right Ctrl
3:29 PM
10/11/2026


Parrot OS 6.4 MATE Security Edition (Phishing Simulation Attack) [Running] - Oracle VirtualBox
Policy Compliance: 100.0%
Average Risk Remediation Time: 5.2 days
Third-Party Risk Assessment: 100%
Patch Compliance Trend
Vulnerability Remediation Time Trend
Security Incidents Trend
Security Monitoring Coverage Trend
Activate Windows
Go to Settings to activate Windows.
Right Ctrl
3:37 PM
10/11/2026

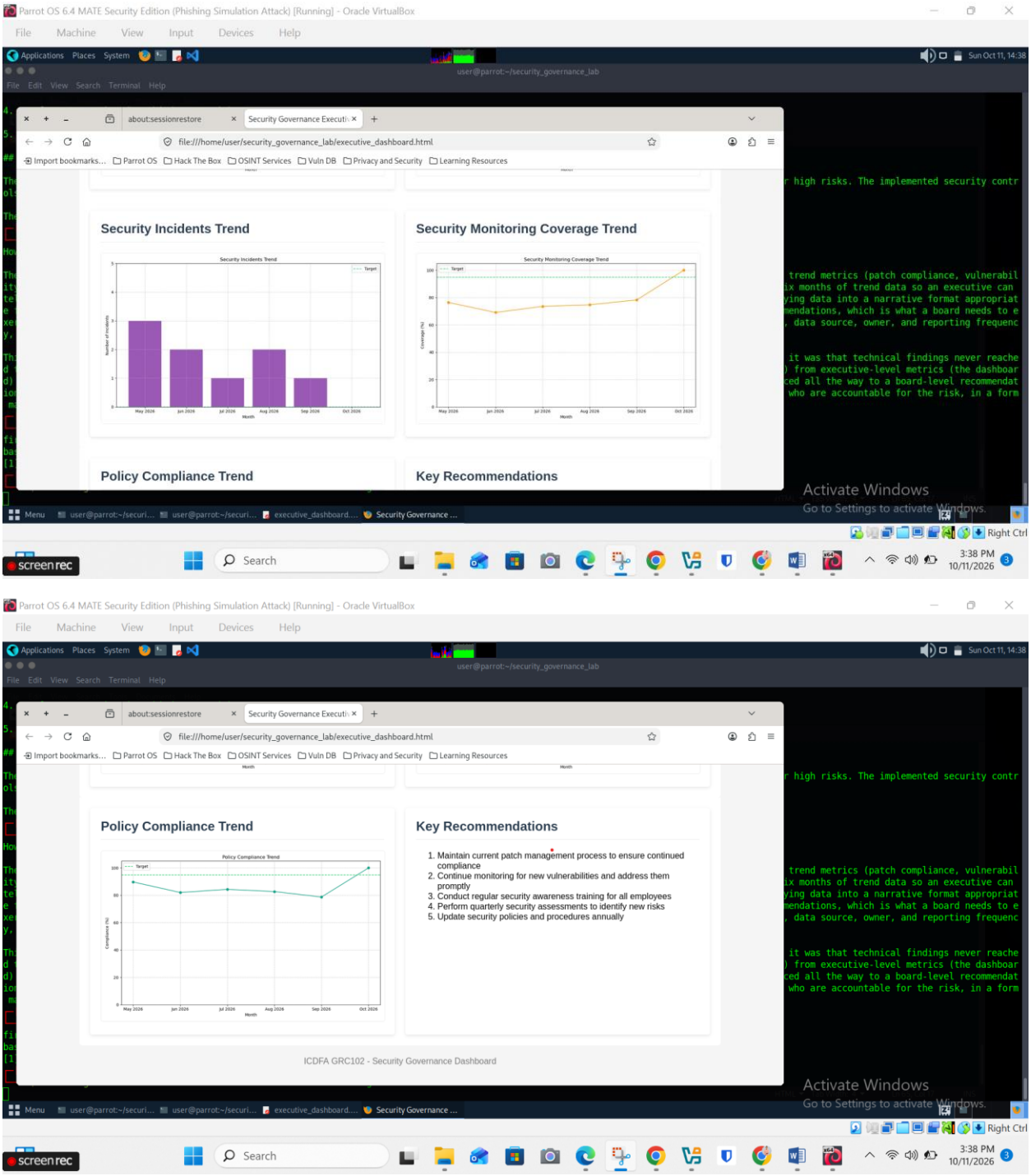


Figure 38. Executive dashboard rendered in a browser showing the five governance e metric charts.

The dashboard file itself, including the chart references and the summarized metric values, is present and correct in the repository and can be opened directly in any browser to view the rendered charts, but the evidence captured in this report for that specific deliverable shows the underlying code and file structure rather than the rendered visual output.

6. Overall Conclusion and Reflection

Taken together, the four parts of this assignment walk through a complete, realistic governance cycle. Part 1 established a technically accurate simulated environment and a governance tracker capable of recording policies, controls, risks, and incidents in one place. Part 2 proved, through a genuine working exploit of CVE-2017-5638, that the baseline governance posture was insufficient, and connected that finding directly to the documented root causes of the real Equifax breach. Part 3 implemented governance controls in response to those findings and, importantly, verified their effectiveness honestly rather than simply assuming they worked, which surfaced the distinction between a control that is tracked in governance records and a control that is technically enforced on the live system. Part 4 built the metrics, dashboard, and board reporting layer that would allow an organization to monitor that same distinction continuously rather than discovering it only during a point in time assessment.

The single thread running through the entire assignment, and the one I would carry into any real governance role, is that governance value comes from verification, not documentation. A policy on file, a control marked as implemented, or a metric showing compliance are only as trustworthy as the mechanism that checks them against what is actually running in production. The Equifax breach is a well-documented, real world example of what happens when that verification step is missing or inconsistent, and this lab reproduced that same gap closely enough, down to the specific CVE involved, that the lesson applies directly rather than only by analogy.

7. References

- National Vulnerability Database. CVE-2017-5638 Detail. National Institute of Standards and Technology. https://nvd.nist.gov/vuln/detail/CVE-2017-5638
- Center for Internet Security. CIS Critical Security Controls. https://www.cisecurity.org/controls
- United States Government Accountability Office. Actions Taken by Equifax and Federal Agencies in Response to the 2017 Breach, GAO-18-559. https://www.gao.gov/products/gao-18-559
- United States House of Representatives Committee on Oversight and Government Reform. The Equifax Data Breach, Majority Staff Report, 115th Congress. https://republicans-oversight.house.gov
- Apache Software Foundation. S2-045 Security Advisory, Apache Struts2 Remote Code Execution. https://cwiki.apache.org/confluence/display/WW/S2-045
- Docker Inc. Docker Compose Documentation. https://docs.docker.com/compose/
- ICDFA GRC102 Practical Lab Assessment Brief, Practical Lab 8: Security Governance Simulation Assessment, provided course material.